

RoboSub CSULA Simulation Software Requirements Specification

Christine (lastname ??)
Katha (idkkkk lastname zzz)
Jason Giang
Vee

September 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Intended Audience	3
1.3	Intended Use	3
1.4	Product Scope	3
1.4.1	In-Scope	3
1.4.2	Out of Scope	3
1.5	Definitions and Acronyms	4
2	Overall Description	5
2.1	User Needs	5
2.2	Dependencies	5
2.2.1	Core Framework Dependencies	5
2.2.2	Communication and Bridge Dependencies	5
2.3	Hardware Requirements	5
3	System Features and Requirements	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	6
4	Other Requirements	7
5	Appendix A: Glossary	8
6	Appendix B: Something	9
7	Appendix C: Another Thing	10

Revision History

Name	Version	Description	Date
Vee	v0.0	Initial draft of the Software Requirements Specification (SRS)	Sep 17, 2026

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe various initial requirements for the RoboSub CSULA Simulation system as of September 17th. This document will define initial baseline requirements, the scope of the simulation system, and potential system architecture. As a living document, the purpose of the document is to provide a shared overview of potential technologies used by the development team, while exploring other potential frameworks for the simulation engine.

1.2 Intended Audience

This document is intended for:

- **Website/Simulation Development Team**
- **Control/Navigation Development Team**
- **Computer Vision Development Team**
- **Instructors**
- **Potential Testers from RoboSubLA Club Team**

1.3 Intended Use

The intended use is to accurately simulate RoboSub controls, navigation, and movement realistically in an underwater environment. The simulation could communicate with the Robotic Operating System (ROS) (I assume ROS is the operating system we will use) to accurately simulate the RoboSub Senior Design Team's sub and possibly the RoboSubLA club's sub.

1.4 Product Scope

1.4.1 In-Scope

- Simulating fluid density, buoyancy vectors, drag, effects on the RoboSub hull, etc.
- Constant/variable water current vectors that the RoboSub must counteract.
- Underwater gates, buoys, trenches, courses to follow, etc.
- Sensor simulation

1.4.2 Out of Scope

- Simulated course is strictly underwater, so air-water physics and surface waves may not be needed
- Structural damage from extreme depths should not need to be simulated
- The environment will focus on course navigation course

1.5 Definitions and Acronyms

- **ROS** — Robotic Operation System
- **CSULA** — California State University, Los Angeles
- **AUV** — Autonomous Underwater Vehicle
- **RoboSub** — Autonomous Robotic Submarine
- **RoboSub Competition** — Yearly national RoboSub competition provided by RoboNation

2 Overall Description

2.1 User Needs

The primary users consist of the RoboSubLA Senior Design team. (Control/Navigation Team, Computer Vision Team, Website/Simulation Team)

The secondary users consist of instructors and the RoboSubLA Club team.

2.2 Dependencies

2.2.1 Core Framework Dependencies

Utilizing the Stonefish C++ library would add the following dependencies:

- **Bullet Physics Library** — Used with Stonefish to provide rigid-body dynamics and collisions.
- **OpenGL Mathematics** — Handles 3D math matrices for sub orientation.
- **SDL2 (libsdl2-dev)** — Framework for application windows, keyboard inputs, and time loop tracking.
- **Freetype (libfreetype6-dev)** — Renders 2D on-screen text.

2.2.2 Communication and Bridge Dependencies

- **Stonefish ROS2 Wrapper** — Provides communication with RoboSub OS.
- **uWebSockets** — Provides a WebSocket server to communicate with the RoboSubLA website
- **Serialization Dependency** — Converts sensor and other data from simulation into JSON for the RoboSubLA website.

2.3 Hardware Requirements

- **Ubuntu 22.04 LTS (Jammy Jellyfish)** — Ubuntu OS provides the least friction regarding compatibility and access to many community libraries, plugins, etc.
- **Graphics Hardware** — Modern graphics hardware that supports OpenGL 4.3 or higher.
- **Processor Hardware** — Modern multi-core CPU capable of running the Stonefish physics engine.

3 System Features and Requirements

3.1 Functional Requirements

- The simulation creates a 3D water environment capable of realistic underwater physics, currents, buoyancy, and drag.
- The simulation loads custom course maps with obstacles such as gates, trenches, and buoys to track, etc.
- The simulation runs from start to finish.
- Data stream to the website, tracking sensor data through JSON.
- The simulation opens a WebSocket, allowing for remote commands or possibly a built-in web dashboard.
- "Invisible" mode that simulates in text-only mode for low-performance hardware and quick, simple tests.

3.2 Non-Functional Requirements

- The simulation handles physics smoothly, performing C++ physics loops running at roughly 100 times per second.
- Instant streaming to web apps. (Under 50 milliseconds)
- Smooth visuals.
- Simple installation and Windows compatibility.

4 Other Requirements

5 Appendix A: Glossary

6 Appendix B: Something

7 Appendix C: Another Thing